Proxmox Storage DRS -- Internals

How the tool works, for whoever changes it

Bernd Zeimetz bernd@bzed.de

2026-09-05

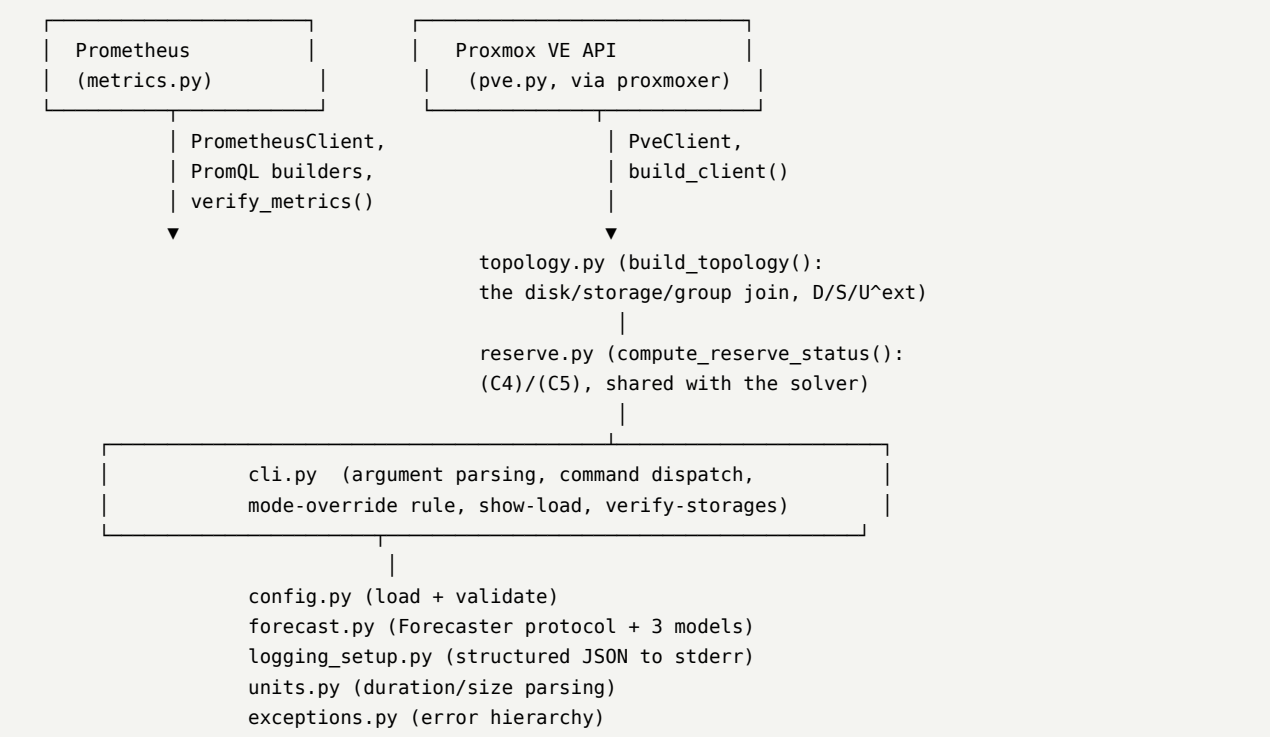
Rendered from `docs/internals/*.md`, SHA-256 `8a4688fa0870f9f4c893753af88b0c860e2482b04dfd9276b92cb3b07d14cc2c`
(`sha256sum --check docs/internals.pdf.sha256` in the repository must succeed).

Contents

What does this page answer? Where does an invocation of `pve-storage-drs` go, which module owns which piece, and how much of `IMPLEMENTATION_PLAN.md`'s section 2 architecture actually exists right now?

The pipeline, as specified and as built

`IMPLEMENTATION_PLAN.md` section 2 describes seven stages: collect, join, gate, solve, cost, order, execute. As of this page, stages 1 (collect) and 2 (join) exist; gate through execute are `IMPLEMENTATION_PLAN.md` section 12 phases 3-9 and are not yet written. Do not take this page as a claim that the whole pipeline runs end to end — `../manual/30-safety-and-status.md` is the authoritative per-command status.



Module layout, as it stands

Module	Responsibility	Plan section
`exceptions.py`	The `DrsError` hierarchy every deliberate failure raises	AGENTS.md section 5
`units.py`	Duration/size parsing (`"24h"`, `"200MiB"`) and human-readable formatting	section 11
`config.py`	Load `/etc/pve/drs.yaml`, jsonschema + semantic validation, the frozen dataclass config model	section 11
`config_schema.json`	The jsonschema structural half of validation	section 11.1
`forecast.py`	The `Forecaster` protocol, `required_range_seconds`, and `quantile`/`seasonal_naive`/`holt_winters`	section 10
`logging_setup.py`	Structured JSON logging to **stderr**	section 2.1 (amended, see `40-cli-and-logging.md`)
`metrics.py`	`PrometheusClient`, PromQL construction, `verify_metrics()`	sections 3.1-3.4
`pve.py`	`PveClient` (built on `proxmoxer`), `build_client()`	section 3.5

Module	Responsibility	Plan section
`topology.py`	`build_topology()`: the disk/storage/group join, `D`, `S`, U^{sext}, (C2) pins	sections 3.5-3.7, 5.1, 5.3 (C2)
`reserve.py`	`compute_reserve_status()`: (C4)/(C5), shared by `show-load` today and the solver later	section 5.3 (C4)/(C5)
`cli.py`	Argument parsing, command dispatch, `--manual`, the mode-override rule, `show-load`, `verify-storages`	section 11.3

Not yet written: `loadmodel.py`, `optimize.py`, `heuristic.py`, `payback.py`, `schedule.py`, `execute.py`.

Why config.py depends on forecast.py

`config.py`'s semantic validation (`_check_forecast_window`) must reject a `window.lookback` too short for the configured `forecast.model` at startup (section 11.1) — that check needs `forecast.required_range_seconds()`. Since `forecast.py` in turn needs `config.ForecastConfig` for its type signatures, the import is deferred to inside the validation function rather than at module level, breaking what would otherwise be an import cycle. See the comment at the top of `config.py`'s `_check_forecast_window` if you touch this.

Where to read next

- `10-configuration.md` — how a YAML file becomes a validated `Config`, and where every default actually lives.
- `20-forecasting.md` — the forecaster protocol and its three implementations.
- `30-metrics.md` — the Prometheus client and the six `verify-metrics` checks.
- `40-cli-and-logging.md` — command dispatch, the `--mode` escalation rule, and why logs go to stderr.
- `50-pve-api.md` — the PVE API client, why it is built on `proxmoxer`, and two things verified against a real cluster.
- `60-topology.md` — the disk/storage/group join, the section 5.1.1 foreign-volume accounting, and the shared (C4)/(C5) evaluator.

How a YAML file becomes a validated `Config`

What does this page answer? What exactly happens between `pve-storage-drs -c foo.yaml plan` and a type-checked `Config` object, and where does each of the section 11.1 validation rules actually live? Describes `proxmox_storage_drs/config.py` and `config_schema.json`.

Resolution order

`resolve_config_path()` implements `IMPLEMENTATION_PLAN.md` section 11's three-source order: `--config` (`cli_path`), then `$PVE_STORAGE_DRS_CONFIG`, then `config.DEFAULT_CONFIG_PATH` (`/etc/pve/drs.yaml`). It also returns `was_explicit`, which `load_config()` uses to decide the error message on a read failure: an explicitly named path that cannot be read is always fatal with no fallback, per section 11, but the *message* additionally points at `config/drs.example.yaml` only when the failing path was the unnamed default — there is nothing useful to suggest when the operator named the path themselves.

The two-stage validation

1. **`_validate_schema()`** loads `config_schema.json` via `importlib.resources` (it ships as package data — see `pyproject.toml`'s `[tool.setuptools.package-data]`) and runs it through `jsonschema`. Every object in the schema sets `additionalProperties: false`: a typo'd key is caught here as a validation error, not silently ignored, which is what keeps section 15.1's knob-to-formula table trustworthy (a key with no formula would otherwise go unnoticed).

2. **`_build_config()`** walks the now-schema-valid raw `dict` into the frozen dataclasses defined earlier in the module, applying every default inline via `.get(key, default)`. These defaults are also what `config/drs.example.yaml` documents and what the manual's `../manual/10-configuration.md` describes — there is deliberately no second copy of a default anywhere else in the codebase.
3. **`_validate_semantics()`** runs the section 11.1 rules jsonschema cannot express — cross-field comparisons, or rules needing a value derived from two independently-optional settings. Each rule is a small `_check_*` function appending to a shared `errors`/`warnings` list (kept separate functions rather than one large one, partly for flake8's `max-complexity` and partly because each is independently testable). `errors` become one `ConfigError` naming every problem found, not just the first; `warnings` are returned to the caller (`ResolvedConfig.warnings`) for `cli.py` to log — section 11.1's `saturation_load` check is the one warning-only rule today.

Every duration or size field is parsed **once**, at this point, via `units.parse_duration_seconds`/`parse_size_bytes`, and stored on the dataclass already in its canonical unit (seconds, bytes) — nothing downstream re-parses a string.

Secrets from the environment

`_build_config()` fills `proxmox.auth.password`/`token_secret` from `PVE_PASSWORD`/`PVE_TOKEN_SECRET` **only when the file's own value is `null`** — a secret actually written in the file is used as-is, never silently overridden by an environment variable a deploy script forgot to unset. This one-directional fallback is what section 11's "keep secrets out of `/etc/pve`" guidance is for: the file can omit the field entirely and rely on the environment, or set it explicitly and take responsibility for the file's own permissions.

What is deliberately *not* checked here

Two section 11.1 rules need live cluster data this module does not have and are therefore **not** part of `config.py`:

- storage ids actually existing in the cluster;
- `execution.source_release.timeout` / `gates.cooldown_per_storage` against a storage's real `saferemove_throughput`.

Both belong to `pve.py`/`topology.py` (not yet written) and `pve-storage-drs verify-storages`, once they exist — see `../manual/30-safety-and-status.md` for current status.

`ResolvedConfig`: what the caller actually gets

`load_config()` returns a `ResolvedConfig`, not a bare `Config`: the resolved `path` and the file's `sha256` travel with it, because `IMPLEMENTATION_PLAN.md` section 11 requires every run to log both — a plan must always be traceable to the exact file that produced it, even though the file is never re-read after startup.

Forecasting: the protocol and its three implementations

What does this page answer? How does `forecast.py` decide how much history a model needs, and what does each of `quantile`/`seasonal_naive`/ `holt_winters` actually compute? Describes `proxmox_storage_drs/forecast.py`.

The one rule, in one place

`IMPLEMENTATION_PLAN.md` section 10.1's table says how much history each model needs, independent of `window.lookback` (the *decision* window). That rule has exactly one implementation: three small pure functions, `_quantile_required_range_seconds`, `_seasonal_naive_required_range_seconds` and `_holt_winters_required_range_seconds`, each called from **two** places — the free function `required_range_seconds()` that `config.py`'s startup validation uses (before any `Forecaster` is constructed), and the matching `Forecaster.required_range()` method on the concrete class. Duplicating the arithmetic

between those two call sites, rather than sharing the pure function, is exactly the kind of "second copy of a rule" AGENTS.md section 5 forbids — a change to the Holt-Winters requirement made in only one of them would silently desynchronize config validation from what the forecaster itself believes it needs.

Why the upper bound, never the point estimate

Every `Forecaster.predict()` returns a `Forecast(point_estimate, upper_bound)`. Nothing in this module enforces that callers use `upper_bound` — that discipline belongs to the caller (the optimizer and the section 7.3 saturation guard, neither written yet). The asymmetry is stated in the module docstring and repeated here because it is easy to get backwards under time pressure: overestimating costs a slightly worse balance, underestimating risks scheduling a mirror onto a storage that is about to saturate.

`QuantileForecaster`

The default. `predict()` takes the `window.quantile`/`window.upper_quantile` percentiles of the raw series with no fitting at all. `_quantile()` implements linear-interpolation percentile matching `numpy.percentile`'s default method, by hand — deliberately not using numpy, since the tool must stay usable with only `requests` + `ruamel.yaml` + `jsonschema` installed (`IMPLEMENTATION_PLAN.md` section 2.1). `horizon` is accepted (to satisfy the `Forecaster` protocol) and immediately discarded: the quantile model has no notion of a horizon-dependent forecast.

`SeasonalNaiveForecaster`

Groups historical samples by hour-of-day (`int((ts // 3600) % 24)`) and takes the median as the point estimate, the configured upper quantile across the same bucket as the bound. `now_epoch_seconds` selects which hour-of-day bucket the *current* moment falls into; `predict()` ignores everything outside that bucket. A bucket with no samples returns `Forecast(0.0, 0.0)` rather than raising, matching `IMPLEMENTATION_PLAN.md` section 4's rule for an idle group's raw quantities: no data is zero, not an error, at this layer (the caller decides whether zero is trustworthy).

`HoltWintersForecaster`

`statsmodels` is an optional dependency (`Suggests`, not `Depends` — see `debian/control`), so it is imported **only inside `predict()`**, never at module level; `debian/tests/import-all` is what would catch a regression here. Two situations fall back to `QuantileForecaster`, both logged at warning level rather than failing silently:

1. fewer than `2 * seasonal_periods` samples in the series (section 10.2's own rule — a fit on too little data is worse than no fit);
2. `statsmodels` is not importable at all.

The upper bound is `point_estimate + residual_z * stdev(in-sample residuals)` — not derived from the model's own confidence interval, which `statsmodels`'s `ExponentialSmoothing.fit()` does not expose directly for every configuration. `residual_z` (`forecast.holt_winters.residual_z`, default `2.0`) is what an operator tunes if this bound turns out too tight or too loose in practice; there is no plan-mandated backtesting gate on it yet (`IMPLEMENTATION_PLAN.md` section 10.2's backtest-validation requirement is not implemented — tracked for the dedicated forecasting phase, section 12 phase 9).

`build_forecaster()`

The one factory function that turns a `ForecastConfig` plus the ambient `window`/`metrics` values into a live `Forecaster` instance. Nothing else in the codebase should construct a `QuantileForecaster`/`SeasonalNaiveForecaster`/ `HoltWintersForecaster` directly once a caller exists that needs one from config — route it through here so the model-name-to-class mapping stays in one place.

The Prometheus client and `verify-metrics`

What does this page answer? How does `metrics.py` talk to Prometheus without ever touching the network in a test, and what does each of the six `verify-metrics` checks actually query? Describes `proxmox_storage_drs/metrics.py`.

`_SessionLike` / `_ResponseLike`: why not `requests.Session` directly

`PrometheusClient.__init__` accepts `session: _SessionLike | None`, a `Protocol` defined in this module with only the one `get(...)` method the client actually calls — not `requests.Session` itself. `.agents/testing.md` forbids any test talking to a real Prometheus; a structural protocol lets a test hand in a small dataclass with a matching `get()` instead of mocking or subclassing the real (network-capable) session. `PrometheusClient()` with no `session` argument still constructs a real `requests.Session()`, which satisfies the protocol structurally — nothing special is needed to make the real thing fit.

`_get()`: one request path, three endpoint shapes

Every one of `instant_query`, `range_query` and `label_values` goes through the private `_get()`, which does the HTTP call, checks the status code, parses JSON, checks Prometheus's own `status: "success"` field, and returns `payload["data"]`. That field's *shape* genuinely differs by endpoint — a dict with a `result` list for `query`/`query_range`, a bare list for `label/<name>/values` — which is why `_get()` is typed `Any` rather than picking one shape; each public method knows which shape it asked for and narrows accordingly. Every failure mode (a transport error, a non-200 status, unparseable JSON, an unsuccessful Prometheus response) raises `MetricsError` from exactly this one place, so a caller catching `MetricsError` around any of the three public methods sees every failure mode uniformly.

PromQL construction

`build_rate_promql()` and `build_quantile_over_time_promql()` are pure string-building functions, deliberately factored out of any query-issuing code so they are unit-testable without a fake session at all — see `IMPLEMENTATION_PLAN.md` section 3.4 for the exact expressions they implement. `raw_metric_name()` is the one place that maps a `RAW_METRIC_FIELDS` entry (`"read_ops"`, ...) to the configured metric name on a `MetricsConfig`, via `getattr` — this is what lets `verify_metrics()` iterate "every configured raw metric" without hand-listing the six field names a second time anywhere in this module.

`verify_metrics()`: six checks, six functions

Each `_check_*` function implements exactly one `IMPLEMENTATION_PLAN.md` section 3.3 numbered check and returns `Findings` (plus, where relevant, the data the report carries forward):

Function	Section 3.3 step	Returns
`_check_metric_names_exist`	1	findings only
`_check_sample_series`	2 + 3 (labels present)	findings, `{metric_name: sample_labels}`
`_check_device_label_collision`	4	one `Finding` or `None` (pure, no I/O)
`_check_coverage`	5	findings, `{DiskKey: coverage_fraction}`
`_check_observed_spacing`	6	findings, `observed_spacing_seconds`

`_check_coverage` and `_check_observed_spacing` both use `metrics.read_ops` as the one representative metric rather than probing all six: a coverage or spacing gap is a property of the underlying Telegraf scrape, not of which of the six raw quantities is read, so probing all six would be six times the Prometheus load for no additional information.

`VerifyMetricsReport.ok` is `True` iff no `Finding` has `level == "error"` — `cli.py`'s `verify-metrics` handler uses exactly this property to decide the process exit code, so there is one definition of "the verification passed."

What `cli.py` does with the report

`_handle_verify_metrics` in `cli.py` constructs a `PrometheusClient` from `resolved.config.prometheus`, calls `verify_metrics()`, and renders either the human form (`[level] message` lines) or, under `--json`, a JSON-serializable dict — `DiskKey` keys are turned into `"vmid:device"` strings there, since JSON object keys must be strings and this module keeps no opinion about output formatting.

Command dispatch, the mode-override rule, and why logs go to stderr

What does this page answer? How does `cli.py` turn `argv` into a dispatched command, what exactly does overriding `--mode` log, and why does structured logging go to stderr instead of the stdout the plan originally specified? Describes `proxmox_storage_drs/cli.py` and `proxmox_storage_drs/logging_setup.py`.

One parser, global options first

`build_parser()` defines every global option (`-c`/`--config`, `--group`, `--mode`, `--json`, `-v`/`--quiet`, `--version`, `--manual`) on the **top-level** `ArgumentParser`, not on the subparsers, which is what makes argparse itself enforce `IMPLEMENTATION_PLAN.md` section 11.3's "global options before the subcommand" for free — there is no hand-written ordering check anywhere. `--help`'s defaults come from the real `default=` values passed to `add_argument`, combined with `argparse.ArgumentDefaultsHelpFormatter`; there is deliberately no second, hand-maintained list of options and defaults anywhere in this module, the manpage, or the manual (AGENTS.md section 8.5).

`main()` handles `--version` and `--manual`/`help` **before** requiring or loading any configuration — printing the version or the manual page must never depend on `/etc/pve/drs.yaml` existing. Only once a real subcommand is present does it call `config.load_config()`.

Command handlers: a dict, not a chain of `if`s

`_COMMAND_HANDLERS: dict[str, CommandHandler]` maps each subcommand name to a function of `(ResolvedConfig, argparse.Namespace, effective_mode) -> int`. Every subcommand not yet implemented (`plan`, `apply`, `show-load`, `explain`, `verify-storages` — see `../manual/30-safety-and-status.md` for which ones that currently is) gets a handler from `_make_not_yet_implemented_handler(name)`, a closure factory rather than a `lambda` with a default-argument trick, because mypy's strict mode cannot infer a bare lambda's parameter types cleanly here — see the comment at that function if you are tempted to simplify it back to a one-liner. `_COMMAND_HANDLERS["verify-metrics"]` is overwritten with the real `_handle_verify_metrics` once `metrics.py` exists to back it. Adding a new implemented command is exactly this: write the handler, assign it into the dict, done — `main()`'s dispatch does not change.

The `--mode` escalation rule

`apply_mode_override(configured_mode, override)` is the entire implementation of section 11.3's rule: moving *down* the ordering `dry-run < confirm < auto` (toward more caution) logs at `INFO`; moving *up* it (toward less caution — removing a barrier the operator's own config set) logs at `WARNING`, naming both values. This is deliberately a small pure-ish function (its only side effect is the log call) rather than inlined into `main()`, so it is unit-testable against a `caplog` fixture without constructing a full CLI invocation.

Why logs go to stderr, not stdout

`IMPLEMENTATION_PLAN.md` section 2.1 originally specified stdout for structured JSON logs. This was amended in the same commit that added `logging_setup.py`: `--json` (section 9.5) emits the plan re-

port on **stdout**, and interleaving log lines with that report on the same stream would make it unparseable by anything downstream. Logging instead goes to **stderr**; a systemd service unit captures both streams into the same journal regardless, so nothing about "captured by journald" is lost. This is the one place in the codebase where the implementation and `IMPLEMENTATION_PLAN.md` disagree with the *original* plan text on purpose — AGENTS.md section 7 rule 2 is why the plan text itself was corrected in the same commit rather than left to silently diverge from the code.

`configure_logging()` is called exactly once, from `main()`, after the `--version`/`--manual` early exits (which must not touch logging configuration at all) and before any config-dependent code runs, so that a config-loading failure is itself logged consistently with everything after it.

`JsonFormatter`: what ends up in one log line

Every field on a `logging.LogRecord` that is *not* one of the standard attributes (computed once, at import time, into `_STANDARD_RECORD_ATTRS`) is folded into the JSON payload — this is what lets any module call `logger.info("...", extra={"event": "gate_decision", "threshold": 0.2})` and have `event`/`threshold` appear as top-level JSON keys with no formatter changes required. `sort_keys=True` keeps output byte-stable for anything that diffs log lines in a test or a log-aggregation pipeline.

`--manual`: preferring `man(1)`, falling back only when necessary

`show_manual()` tries `man pve-storage-drs` first, via `subprocess.run` rather than `os.execvp`: `man(1)` itself detects a non-tty stdout and disables its pager automatically, which is what satisfies "never answer with a URL alone" (AGENTS.md section 8.5) whether the invocation is interactive or piped — there is no need for this module to duplicate that tty detection. `_fallback_manual_text()` is reached only when `man(1)` itself is missing or has no entry (an uninstalled source checkout, or a minimal system with no man-db): it walks up from `cli.py`'s own file location looking for `man/pve-storage-drs.1.md` in a source tree, and only if that also fails prints a short message naming the real installed-package path and the in-tree file — a local, actionable path, never a bare URL.

The Proxmox VE API client

What does this page answer? Why is `pve.py` built on `proxmoxer` instead of a hand-rolled ticket/CSRF client, and what does each of its methods actually call? Describes `proxmox_storage_drs/pve.py`.

Why `proxmoxer`

`IMPLEMENTATION_PLAN.md` section 3.5 explains the decision in full; the short version is `proxmoxer`'s **backend abstraction**. The same `ProxmoxAPI` attribute-chaining interface (`api.nodes(node).qemu(vmid).config.get()`) is available over plain HTTPS (what `build_client()` uses today) or over SSH, either shelling out to the system's own `ssh+pvesh` (`openssh`) or with an in-process client (`ssh_paramiko`). A deployment that cannot open the API port to the management host becomes a different keyword argument in `build_client()`, with no change anywhere else — not to `PveClient`'s methods, and not to any of their callers.

`build_client()`: the one place auth is assembled

`config.py`'s `ProxmoxConfig` keeps `verify_ssl` (bool) and `ca_file` (path-or-`None`) as two separate knobs, because that is how an operator thinks about them; `requests` (and so `proxmoxer`'s https backend) wants one value, a bool or a CA-bundle path, passed as `verify`. `build_client()` is the one place that reconciles the two. It is also the one place `proxmox.auth.token_id`'s `user@realm!tokenname` format is split into `proxmoxer`'s separate `user`/`token_name` keyword arguments — never duplicate that parsing anywhere else.

Every failure `ProxmoxAPI(...)` itself can raise (`proxmoxer.AuthenticationError`, or a `requests.RequestException` if the host is simply unreachable) is caught here and re-raised as

PveApiError, so every caller of build_client() and every PveClient method has exactly one exception type to catch.

PveClient: one method per section 3.5 endpoint

Every method is a single API call, wrapped through the private _call() helper, which is the one place proxmoxer.ResourceException (HTTP-level API errors) and requests.RequestException (transport failures proxmoxer's https backend does not wrap itself) both become PveApiError. No method caches anything, and the per-run topology cache belongs to topology.py (section 3.5), which is also where the bounded thread pool for the O(VMs) config fetch lives (60-topology.md, REVIEW.md P-02).

_call() does retry exactly one failure mode: proxmoxer.AuthenticationError gets one reauthenticate-and-retry cycle (REVIEW.md P-01) before becoming a PveApiError, via a reauthenticate callback build_client() wires in — a full fresh login through _build_api() again, not a reuse of the rejected ticket. This is not a general retry policy (a ResourceException or a transport failure still fails immediately, on the first attempt); it exists specifically because a ticket can expire for reasons with nothing to do with the call that hits it — a long apply --confirm wait, a suspended process, a clock jump — and proxmoxer's own lazy per-request renewal only notices the age of its *own* clock, not whether the server-side ticket actually outlived a gap that long. A test double built with a bare PveClient(fake_api) (no reauthenticate=) gets none of this — it behaves exactly as it did before P-01, which is what every existing fake in tests/unit/fakes.py relies on.

build_client() also applies proxmox.ticket_refresh_seconds to the constructed session, best-effort: proxmoxer 2.x has no constructor argument for a password/ticket auth's refresh interval (it is a hard-coded renew_age = 3600 class attribute on ProxmoxHTTPAuth, checked lazily on whichever request happens to run after it elapses), so _apply_ticket_refresh_seconds() reaches into api._backend.auth — undocumented proxmoxer internals, not its public interface — to override that instance's renew_age after login. Deliberately narrow: it only ever *tightens* proxmoxer's own schedule, never replaces its refresh logic, and if a future proxmoxer version reshapes this (or the backend is not https, or the auth is API-token, which has no ticket at all), the getattr/hasattr guards make it a silent no-op — the P-01 reauthenticate-and-retry above still catches the case this can't reach.

storage_definitions() deliberately uses GET /storage (the list form), never the singular GET /storage/{id}, even though only one storage's config is needed in some callers. This was verified empirically against a real PVE 9.2.11 cluster during development ([[dev-cluster-access]] in the maintainer's notes, not reproduced here): an API token granted only Datastore.Audit can list every storage's full config — including saferemove/saferemove_throughput — via GET /storage, but the identical data via GET /storage/{id} for the same storage returns 403 Forbidden (Datastore.Allocate). The single-item route apparently backs an edit-UI flow gated by the ability to change the config, not merely read it. Since the list form returns the same per-storage object for every storage in one call, there is no reason to ever call the singular form, and using it would quietly force a Storage DRS token to hold more privilege than the tool actually needs. IMPLEMENTATION_PLAN.md section 3.5 was corrected to match once this was found — see that section's note.

storage_content() returned an empty list — not an error — on every storage, node and content-type filter tried, until the token also held Datastore.Allocate on that storage. This was genuinely surprising and worth calling out precisely because it is a *silent* false negative: Datastore.Audit alone gives a clean 200 with {"data": []}, which looks exactly like "this storage legitimately has no tracked content" rather than "the caller cannot see it." It was not guessed at — see IMPLEMENTATION_PLAN.md section 3.5's note, which records the exact before/after (0 entries with Audit only, 37 and 1 entries respectively for two real storages once Datastore.Allocate was added) on the same live cluster. **The practical consequence: pve.py's own "keep the token to Audit-only" goal for storage_definitions()**

does not extend to `storage_content()` — a Storage DRS token genuinely needs `Datastore.Allocate` (bundled with `Datastore.Audit` in one role) on every storage it manages, or `topology.py` will silently compute U^{sext} (section 5.1.1) as if every storage were empty of untracked volumes, with no error to notice by. `docs/manual/00-installation.md` states the exact minimum ACL grant an operator needs; this is not a place to economize on privilege out of a preference that turned out not to match reality.

Two more things about Proxmox's ACL model this surfaced, both now reflected in the manual's token-setup instructions: **an API token's effective permission is the *intersection* of its owning user's permissions and whatever is granted to the token itself** when privilege separation is on (the default) — granting a role to only one of the two has no effect, both need it; and the grant that was confirmed to work was made explicitly at each `/storage/{id}` rather than only at the parent `/storage` path, so that is what the manual instructs, rather than relying on propagation that was not cleanly isolated as working or not.

`move_disk()`: the one and only bytes/s -> KiB/s conversion

Section 9.2 is explicit that `bwlimit`'s bytes/s-to-KiB/s conversion happens "at the call site and nowhere else." `PveClient.move_disk()` is that call site: every caller in this codebase, present and future, passes `bwlimit_bytes_per_sec` in the same unit `migration.bwlimit_bytes_per_sec` already uses, and the division by 1024 (rounded, since the API wants an integer) happens exactly once, inside this method. `format` defaults to `None` and is only ever forwarded when a caller explicitly passes one — `move_disk` without `format=` preserves the source format, which is what section 3.5 requires by default.

Building the disk/storage/group model

What does this page answer? How does `topology.py` turn `pve.py`'s raw API responses into the per-group `D`/`S` sets the rest of the engine needs, and where does each `IMPLEMENTATION_PLAN.md` section 5.3 (C2) pin condition actually get decided? Describes `proxmox_storage_drs/topology.py` and `proxmox_storage_drs/reserve.py`.

`D` means every disk this module places, pinned or not

This is stated at the top of `topology.py`'s own docstring because it is the single easiest thing to get backwards: `D` (section 5.1) is *every* disk this module puts into a group's `Group.disks`, whether or not (C2) pins its placement. A disk this module never sees at all — a stopped VM excluded by `exclude.running_only`, or a disk whose current storage is not in any configured group — is what "foreign" (U^{sext}, section 5.1.1) means. Config-excluded disks (`exclude.vmids`/`exclude.disks`/tags) are *not* foreign: (C2) pins them into `D` specifically so their bytes still count in (C4)/(C5) and their fragmentation toward κ (section 3.6, "Pinned disks are modelled, not ignored"). An earlier draft of section 5.1.1 listed config-excluded disks as foreign, which directly contradicted (C2) — that self-contradiction was found and fixed in the same commit that first implemented this join; see that section's note if you need the history.

One pass, in the order section 3.5 lists

`build_topology()` fetches everything it needs exactly once, in five stages (`_fetch_cluster_data`, then a loop over `client.vm_resources()` calling `_collect_vm_disks` per VM, then `_build_storages`):

1. `storage_definitions()` (the list form — see `50-pve-api.md`), validated against the configured `groups` (`_validate_group_storages`): a storage that does not exist, or that lacks `images` in its content types, is a fatal `TopologyError` (section 11.1: "storage ids exist in the cluster"). A storage that exists but is not marked `shared` is a warning, not a fatal error — plausible, if unusual, for a single-node group.
2. `storage_resources()` once, to pick one active node per storage (`_pick_active_node`, preferring one reporting `status: "available"`).

3. `storage_content()` and `storage_status()` per group storage.
4. `vm_config()` and `vm_snapshots()` per considered VM. A VM is *not* fetched at all when `exclude.running_only` is set and it is stopped — the one case `cluster/resources`'s own fields can decide without a config fetch. Every other VM is fetched, including config-excluded ones: whether a specific disk turns out to be "ungrouped" is only knowable after seeing which storage it is actually on, and a config-excluded VM's disk still needs its size for (C4)/(C5) (previous section).

   This is the one step that runs across a bounded thread pool (`config.proxmox.read_workers`, REVIEW.md P-02): section 3.5's whole point is that `GET .../qemu/{vmid}/config` has no batch form, so for a several-hundred-VM cluster this is the read phase concurrency actually helps. It is split into `_fetch_vm()` (network I/O only, run by the pool, one call per considered VM) and `_join_vm_disks()` (pure — the same function the old sequential `_collect_vm_disks()` was renamed from, unchanged in what it computes). `ThreadPoolExecutor.map()` returns each VM's fetch in the same order `client.vm_resources()` listed it, even though the fetches themselves complete in whatever order the pool schedules them, so the join phase runs single-threaded and in the original order — `disks_by_group`, `referenced_volids` and `warnings` end up byte-for-byte identical to a fully sequential run regardless of `read_workers` or of thread scheduling, and never need a lock. `PveClient` itself may reauthenticate mid-fetch if several threads hit an expired ticket at once (`50-pve-api.md`'s P-01 note); that reauthentication is what its own lock serializes, not this join.
5. Non-QEMU resources (`type != "qemu"`) are skipped outright — this tool never touches LXC containers, and section 3.5's read/write paths are qemu-only throughout.

The `lock` field comes from `vm_config()`, not a separate call

Section 9.3's own pseudocode says `lock` is readable from either `/qemu/{vmid}/config` or `/status/current`. `_collect_vm_disks` reads it from the config response already being fetched for the disk join, so planning-time lock detection costs no extra API call. `/status/current` is still needed later, but only immediately before a specific move (section 9.2's pre-move re-validation, `execute.py`, not yet written) — never here.

Pin priority: `_pin_reason()`

One function, checked in the fixed order section 5.3 (C2) lists its conditions: config exclusion (VM-level, then `exclude.disks`), a real snapshot or an unreferenced companion volume (section 3.7, `_disk_snapshot_or_orphan_reason` — the `"current"` entry `GET .../snapshot` always returns, verified on a live cluster, is filtered out before counting), a VM lock, then an `unusedN` disk when `exclude.include_unused_disks` is false. A disk gets at most one reason; the first that applies wins, matching how an operator would explain it.

Sizes: content is authoritative, config is the fallback

`_resolve_disk_size_and_format` looks up the volume in that storage's content listing first (section 3.5: authoritative for what is actually allocated) and falls back to parsing the VM config's own `size=` only when the volume is missing from content — logged as a warning naming the disk, since assume-thick-provisioning sizing from a stale or unlisted config value is a real, if rare, source of drift. Two independently verified reasons this fallback path is not merely defensive: the `Datastore.Allocate`-vs-`Audit` privilege gap (`50-pve-api.md`), and an `unusedN` volume, deleted directly on the storage backend outside Proxmox (confirmed with the cluster's operator), that PVE's own config still referenced and that PVE itself never noticed or refused to boot the VM over — `IMPLEMENTATION_PLAN.md` section 3.6's note. `_parse_pve_config_size_bytes` parses PVE's own config-file size suffixes (`512G` meaning binary GiB, no explicit `i`) — deliberately not `units.py`'s parser, which is for *this project's* config file, a different and coincidentally similarly-shaped format.

U^{ext}: everything not referenced

Every disk actually placed into a group's `D` records its volid in `referenced_volids[storage_id]` as it is processed. `_build_storages` then sums the size of every content entry on that storage whose volid was never referenced — orphans, templates, other groups' foreign volumes, and disks of VMs this run never fetched (stopped-and-excluded, or ungrouped) all fall out of this one computation with no special-casing per category, which is exactly section 5.1.1's definition read literally.

`reserve.py`: one (C4)/(C5) evaluator, shared

`compute_reserve_status()` is deliberately its own module, not a method on `Storage`: the solver (`optimize.py`/`heuristic.py`, not yet written) will need to evaluate the identical formula against a *candidate* assignment, not only the current one, and `pve-storage-drs show-load`'s reporting needs it against the current assignment today. Both call the same function (AGENTS.md section 5) — `show-load`'s use of it is already exercised end-to-end (see `cli.py`'s `_render_show_load_human`/`_json`). The section 14 worked example's initial state (`tests/unit/test_reserve.py`) is the proof this reproduces the plan's own arithmetic exactly, not merely a self-consistent unit test.